SwarmProtocol — CMPE 485 Project Report

CMPE 485 Term Project Report (Option B)

Alperen Garip

May 2026

SwarmProtocol

A 3D real-time survivor-style arena game built around orthogonal architecture and benchmark-driven performance work.

CMPE 485 Term Project (Option B) — Alperen Garip — Unity 6.3 (URP) — macOS Standalone

Source code: https://github.com/AlperenGarip/CmpE485-Projects/tree/main/SwarmProtocol

---

1. Introduction and Motivation

1.1 What SwarmProtocol Is

SwarmProtocol is a single-player, real-time, 3D arena survival game. The core loop is short and self-contained: the player spawns at the center of a fixed arena and must survive a five-stage run, each stage a timed wave of escalating enemies. All weapons

fire automatically; the player controls only movement (camera-relative WASD) while an auto-aim system selects the nearest enemy target. Experience drops from killed enemies fill an XP bar; on level up, the game pauses and presents three upgrade choices (new weapons, level-ups for existing weapons, or passive items). Gold coins fund nothing in the current build but are tracked and displayed for a planned meta-progression layer. The run ends in victory when the Stage 5 timer expires, or in defeat when the player's health pool reaches zero.

1.2 Inspiration and Distinguishing Decisions

The design lineage is unmistakable: *Vampire Survivors* (poncle, 2022) defined the genre, and SwarmProtocol takes its auto-fire, build-stacking, slot-cap, and timed-wave skeleton from it. Three deliberate departures separate the projects:

1. **Dimensionality.** *Vampire Survivors* is 2D top-down with sprite-based enemies and a fixed orthographic camera. SwarmProtocol is fully 3D with an orbiting Cinemachine third-person camera, which requires real navigation (NavMesh) for enemies and a camera-relative input model for the player.
2. **Treasure-chest presentation.** Chests in *Vampire Survivors* deliver instant rewards; SwarmProtocol uses a tiered slot-machine reveal: a tier is rolled (1–5), the corresponding number of columns spin with accelerating-then-decelerating cadence, and the locked combination is then evaluated for jackpots (pairs, triples, four-of-a-kind Overcharge, five-of-a-kind forced Weapon Evolution).
3. **Architectural focus.** Rather than racing to maximum content, SwarmProtocol is organized so that adding the *next* enemy, weapon, or passive is a content addition (a ScriptableObject asset and at most one short subclass), not an engine modification. This is the central thesis of the project.

1.3 Course Context

The project is submitted under CMPE 485 Term Project, Option B (a simple game with an architectural and performance emphasis). The deadline is May 10, 2026. Deliverables include the live build, a six-minute presentation, this written report, and the public source-code repository.

---

2. Game Design

2.1 Core Loop and Win/Lose Conditions

The high-level loop is:

```
Main Menu  -->  Stage 1 ... Stage 5  -->  Victory
   ^                |                       (or)
   |                +--->  Level-Up screen (timeScale=0, pick 1 of
3)
   |                |
   |                +--->  Chest Open (timeScale=0, slot machine)
   |                |
   |                +--->  Pause Menu (timeScale=0)
```

```
|                |
+----------  Game Over  <---  player HP reaches 0
```

The two terminal conditions are explicit and unambiguous: **win** = the Stage 5 countdown reaches zero; **lose** = the player's HP reaches zero. There are no escape options mid-run other than quitting through the pause menu, which returns to the main menu without saving progress.

2.2 Mechanics

Weapons (six concrete fire strategies): Pulse Rifle (rapid-fire projectile), Scattergun (shotgun spread), Power Fist (short-range melee strike), Nduja Flamethrower (a special pickup-driven flamethrower override), Flail (orbital projectile that circles the player), Poison Aura (continuous area damage). Each weapon is a `WeaponDataSO` asset paired with a `FireStrategySO` that defines how it actually fires; the same generic `StrategyWeapon` host runs all of them. The player can hold up to six weapons simultaneously, and any held weapon can be levelled up multiple times via upgrade picks or chest rewards.

Enemies (four archetypes): Swarmer (low health, fast, contact damage), Ranged (keeps distance, fires projectiles), Tank (high health, slow, heavy contact damage), Elite (boss-tier, drops a treasure chest on death). All four share the same generic `BehaviorDrivenEnemy` host plus a per-type `EnemyBehaviorSO` asset.

Passives (ten items): Spinach (+Might), Bracer and Leather Armor (+Armor), Heart Up (+Max HP), Wings (+Move Speed), Candelabra (+Area), Clover Leaf (+Luck), Candy Box (+Recovery per second), Attracter (+Magnet radius), Skull O' Maniac (+Curse). The Curse stat is a risk-reward modifier: higher Curse increases enemy spawn rate and per-enemy multipliers but also boosts drop rates. The player holds up to six unique passives; each can be ranked up multiple times.

Stages (five timed waves): Each stage is defined by a `StageDataSO` that lists spawn brackets (enemy mix, interval, count) timed against a countdown. Stage 5 is the final stage; surviving its timer triggers the Victory screen.

Treasure-chest slot machine: Killing an Elite drops a chest pickup. Collecting it pauses gameplay and opens the chest UI, which proceeds in three phases:

1. *Tier reveal* — a tier from 1 to 5 is rolled, weighted by player Luck. The corresponding number of slot columns is enabled and a "Tier N Chest!" banner pops in.
2. *Spin* — each enabled column ticks through a random pool of upgrades at an accelerating-then-decelerating interval, then locks on its predetermined result. Columns lock in sequence to build anticipation.
3. *Jackpot evaluation* — a `JackpotProcessor` inspects the locked combination:
   - All distinct: each item is applied once (standard reward).
   - Pair: the matching upgrade is applied with a +1 level bonus.
   - Triple: applied with a +2 level bonus.
   - Four-of-a-kind: applied, plus an Overcharge buff (fire-rate + damage multiplier for 60 s).
   - Five-of-a-kind: bypasses normal evolution requirements and triggers `WeaponEvolutionSystem.ForceEvolve()` on the matching weapon.

2.3 Design Choice Space

A handful of architectural decisions shaped the code, each with a defended trade-off:

Concern	Option chosen	Rejected alternative	Reason
Global state coordination	Thin FSM + EventBus	God-class `GameManager` that knows every UI panel	New state-aware UI is one new file with one subscription; the manager never grows
Enemy / weapon definition	Strategy as ScriptableObject	One `MonoBehaviour` subclass per type	Designers can author new content without recompiling, and behavior + data live in inspectable assets
Stat stacking for passives	`IStatProvider` decorator chain	Flat `Dictionary<StatType, float>` summed in place	Each passive is an independent layer that can be added or removed without touching combat code
Enemy spatial queries	Cached `ActiveEnemyRegistry` + `sqrMagnitude`	`Physics.OverlapSphere` / `FindGameObjectsWithTag`	Zero GC alloc, no broad-phase overhead, no collision-matrix dependency
Frequent spawn/ despawn	Generic `ObjectPoolManager` + `IPoolable`	`Instantiate` / `Destroy` per shot	Eliminates GC.Alloc per shot and the periodic `GC.Collect` spikes that follow

2.4 Libraries and Assets

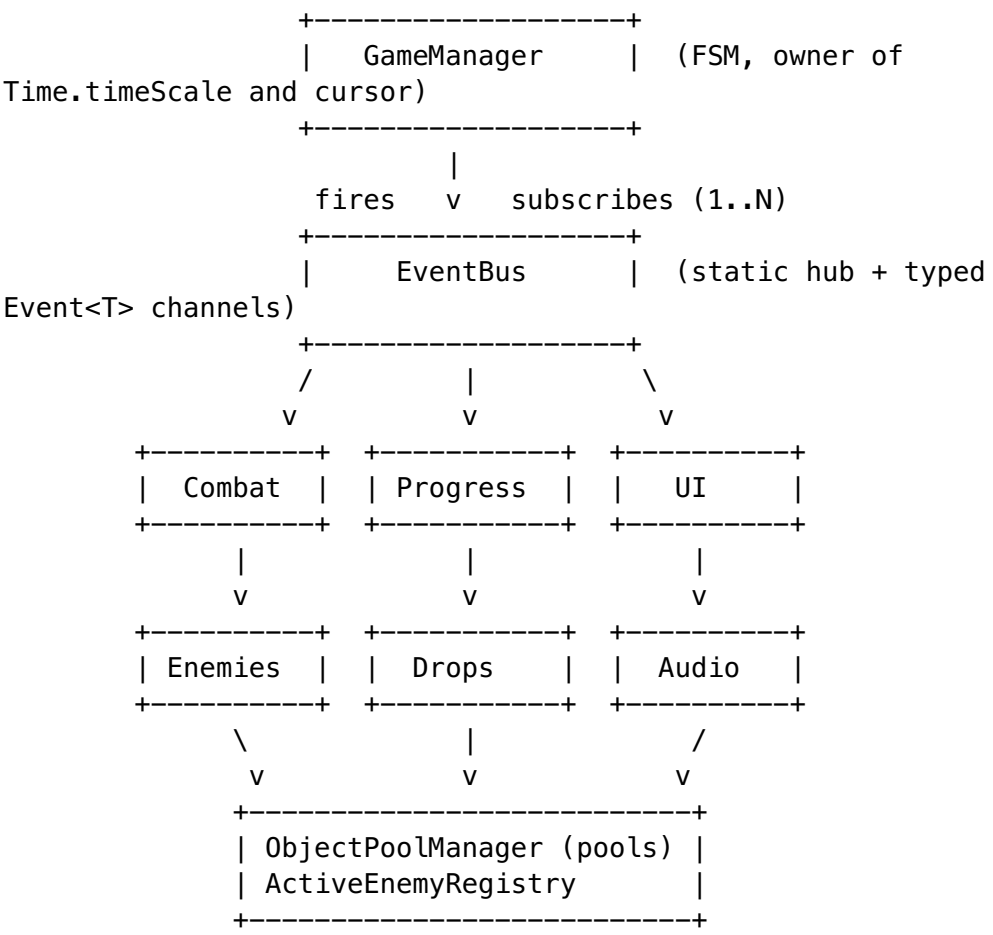
- **Engine:** Unity `6000.3.9f1` (Unity 6.3) with the Universal Render Pipeline.
- **Camera:** Cinemachine third-person rig with orbital free-look.

- **Input:** Unity Input System (new).
- **UI text:** TextMeshPro.
- **AI navigation:** Built-in NavMesh.
- **Content generation:** Unity AI tooling for sprite icons (weapons, passives) and sound effects/music. Generated assets are imported as standard sprites/ AudioClips and used through the project's own audio service.

---

3. Architecture Overview

3.1 Module Map



Systems communicate by raising events on `EventBus`; only `GameManager` and `ObjectPoolManager` are touched directly (as singletons).

3.2 Design Patterns Used

Pattern	Location	Role
Singleton	`GameManager`, `ObjectPoolManager`, `ActiveEnemyRegistry`, `AudioService`, `GoldManager`	Single point of access for global services

Pattern	Location	Role
Observer / EventBus	`EventBus.cs`, `Event<T>`	Decoupled cross-system notifications
Strategy (as ScriptableObject)	`FireStrategySO`, `EnemyBehaviorSO` and subclasses	Swappable behavior owned by data assets
Object Pool	`ObjectPoolManager` + `IPoolable`	Reuse of projectiles, enemies, pickups
Decorator	`IStatProvider`, `BaseStatProvider`, `PassiveDecorator`	Stackable passive bonuses
State Machine (FSM)	`GameManager` (game states), `EnemyAI` (per-enemy states)	Clean transitions, explicit state-aware behavior
Registry	`ActiveEnemyRegistry`	Centralized canonical list of alive enemies
Facade	`PlayerStats` properties	Hide decorator-chain lookups behind a friendly API

3.3 Namespace Organization

```
SwarmProtocol.Core              GameManager, GameState, EventBus,
ObjectPoolManager,
                                ActiveEnemyRegistry,
ServiceLocator
SwarmProtocol.Events            Event<T> + event structs
(PassiveAppliedEvent,
                                StatsChangedEvent,
OverchargeTriggeredEvent, ...)
SwarmProtocol.Player            PlayerController, PlayerHealth,
PlayerStats, PlayerXP
SwarmProtocol.Combat            WeaponManager, WeaponBase,
ProjectileBase, DamageSystem
SwarmProtocol.Combat.Strategies FireStrategySO + Aura/Orbital/
RapidFire/Shotgun/
                                Melee/Nduja subclasses
SwarmProtocol.Enemies           EnemyBase, BehaviorDrivenEnemy,
EnemyNavigation,
                                EnemyBehaviorSO + per-type
subclasses
SwarmProtocol.Stats             IStatProvider, BaseStatProvider,
PassiveDecorator,
                                StatType
SwarmProtocol.Progression       StageManager, UpgradeManager,
GoldManager,
                                WeaponEvolutionSystem
SwarmProtocol.Chest             ChestRewardResolver,
JackpotProcessor, OverchargeManager
SwarmProtocol.UI                HUDController, LevelUpUI,
```

```
ChestOpenUI,
                                SlotMachineColumn, VictoryUI,
GameOverUI, ...
SwarmProtocol.Audio             AudioService, AudioLibrarySO,
SfxId
SwarmProtocol.Tools             BenchmarkRunner
```

The boundaries are intentionally coarse so that the unit of orthogonality (a weapon, an enemy, a passive, a UI panel) lives entirely inside one namespace.

---

4. Implementation Details

4.1 Game State FSM and EventBus

`GameManager` is a small finite-state machine with the following states: `Menu`, `Playing`, `LevelUp`, `ChestOpen`, `Paused`, `StageTransition`, `GameOver`, `Victory`. Transitions are encapsulated in methods (`StartGame()`, `EnterLevelUp()`, `ResumeFromChestOpen()`, etc.). Each transition sets `Time.timeScale` (1 for `Playing`, 0 for the modal-pause states) and the cursor lock state, then fires a single `EventBus.OnGameStateChanged(newState)` event.

Every state-aware system subscribes independently. For example, `VictoryUI`:

```
private void Awake()     { EventBus.OnGameStateChanged +=
        OnGameStateChanged; }
private void OnDestroy() { EventBus.OnGameStateChanged -=
        OnGameStateChanged; }

private void OnGameStateChanged(GameState newState)
{
    bool show = newState == GameState.Victory;
    victoryPanel?.SetActive(show);
    if (show) PopulateStats();
}
```

Adding a new state-aware UI panel does not require any edits to `GameManager`. Adding a *new* state requires one enum value and one transition method on `GameManager`, but all existing subscribers continue to work unchanged.

4.2 Strategy as ScriptableObject

The most-extended pattern in the codebase. Concrete classes:

- `FireStrategySO` (abstract) — `AuraStrategySO`, `RapidFireStrategySO`, `ShotgunStrategySO`, `MeleeStrategySO`, `OrbitalStrategySO`, `NdujaStrategySO`.
- `EnemyBehaviorSO` (abstract) — one subclass per enemy archetype.

A `WeaponDataSO` carries the strategy reference and per-weapon stats (base damage, range, fire rate, knockback). At runtime, the generic `StrategyWeapon` host calls `fireStrategy.Execute(fireContext)`; the strategy reads weapon stats from the context and runs its algorithm.

Example: `AuraStrategySO.Execute` (abridged):

```
public override void Execute(FireContext ctx)
{
    using (_markerExecute.Auto())
    {
        float range    = ctx.WeaponData.range *
        ctx.AreaMultiplier;
        float sqrRange = range * range;
        _hitBuffer.Clear();

        ActiveEnemyRegistry.Instance?.GetEnemiesInRange(
            ctx.FirePoint.position, sqrRange, _hitBuffer);

        float damage = (ctx.WeaponData.baseDamage +
        ctx.LevelDamageBonus)
                       * ctx.OverchargeDamageMultiplier;
        foreach (var enemy in _hitBuffer)
            DamageSystem.ApplyDamage(enemy, damage,
        ctx.DamageMultiplier,
                ctx.CritChance, ctx.FirePoint.position,
                ctx.WeaponData.knockbackForce,
        ctx.WeaponData.knockbackDuration);
    }
}
```

To add a new weapon type, the engineer writes one new `FireStrategySO` subclass (one short file) and a designer authors as many `WeaponDataSO` assets as desired against it. No changes to `WeaponManager`, `StrategyWeapon`, or any other consumer.

4.3 IStatProvider Decorator Chain

The player's stat sheet is not a flat dictionary updated in place. It is a chain of `IStatProvider` decorators rebuilt every time a passive is applied:

```
public interface IStatProvider
{
    float Get(StatType stat);
}

public class BaseStatProvider : IStatProvider
{
    private readonly Dictionary<StatType, float> _values;
    public BaseStatProvider(Dictionary<StatType, float> v) =>
        _values = v;
    public float Get(StatType s) => _values.TryGetValue(s, out
        var v) ? v : 0f;
}
```

```
public class PassiveDecorator : IStatProvider
{
    private readonly IStatProvider _inner;
    private readonly StatType _stat;
    private readonly float _value;
    private readonly bool _isPercentage;
    // ... constructor ...
    public float Get(StatType stat)
    {
        float v = _inner.Get(stat);
        if (stat != _stat) return v;
        return _isPercentage ? v * (1f + _value) : v + _value;
    }
}
```

`PlayerStats.RebuildChain()` constructs a fresh `BaseStatProvider` from the selected character's base values, then wraps it in one `PassiveDecorator` per applied passive:

```
IStatProvider chain = new BaseStatProvider(baseValues);
foreach (var passive in _appliedPassives)
    chain = new PassiveDecorator(chain, passive.statType,
                                 passive.value,
        passive.isPercentage);
_chain = chain;
```

Calling `chain.Get(StatType.Might)` walks the chain bottom-up, with each decorator either contributing its bonus (if its stat matches) or passing through. A new stat is added by extending the `StatType` enum; every existing passive, every base value entry, and every consumer continues to work unchanged. The chain is rebuilt rather than mutated to keep the structure immutable per snapshot — there is no risk of an in-place edit corrupting a partially read value.

4.4 ActiveEnemyRegistry and `sqrMagnitude` Queries

`ActiveEnemyRegistry` is a singleton holding a `List<EnemyBase>` of every alive enemy. Enemies register on spawn (in `EnemyBase.Awake` or `IPoolable.OnSpawn`) and unregister on death/despawn. Two public query methods serve every spatial lookup in the game:

```
public void GetEnemiesInRange(Vector3 center, float sqrRange,
        List<EnemyBase> results)
{
    results.Clear();
    for (int i = _activeEnemies.Count - 1; i >= 0; i--)
    {
        var enemy = _activeEnemies[i];
        if (enemy == null) { _activeEnemies.RemoveAt(i);
        continue; }
        if ((enemy.transform.position - center).sqrMagnitude <=
        sqrRange)
            results.Add(enemy);
    }
```

```
}

public EnemyBase GetNearest(Vector3 origin)
        { /* analogous, tracks bestSqr */ }
```

Two design points are worth calling out. First, the caller pre-computes `sqrRange = range * range` once and passes it in; the inner loop is a vector subtract, three multiplications, and one comparison per enemy. There is no square-root call (the comparison is in squared space), no physics broad-phase, and no GC allocation. Second, the loop iterates backwards and removes stale entries lazily so that destroyed enemies still on the list cannot crash a query mid-frame.

The registry is used by every AOE weapon strategy (`AuraStrategySO`, `OrbitalStrategySO`, `NdujaStrategySO`, `MeleeStrategySO`), by `PlayerController.FindNearestEnemy` for auto-aim, and by `OrologionPickup.FreezeAll`. Section 8 quantifies the speedup.

4.5 Generic ObjectPoolManager

`ObjectPoolManager` is a singleton that maintains a per-prefab pool, created lazily on first request. The interface is:

```
public T Get<T>(T prefab, Vector3 position, Quaternion rotation)
        where T : Component, IPoolable;
public void Return<T>(T instance) where T : Component, IPoolable;
```

The first `Get<T>(prefab, ...)` for a previously unseen prefab creates a new pool keyed by the prefab's instance ID. Pooled components implement `IPoolable`:

```
public interface IPoolable
{
    void OnSpawn();    // reset state, enable colliders, restart
        agents
    void OnDespawn();  // clear caches, disable agents, drop
        references
}
```

Every projectile (`ProjectileBase`), every enemy (`EnemyBase`), every pickup (`XPGem`, `GoldCoin`, `TreasureChest`) implements `IPoolable`. The pool never destroys a GameObject during a run; it simply deactivates and reuses.

The NavMesh interaction deserves a note: `OnDespawn` stops and disables the `NavMeshAgent` rather than disabling the whole GameObject hierarchy, which would otherwise corrupt the NavMesh tracking. `OnSpawn` re-enables the agent and pushes a new destination. This is the only non-trivial lifecycle requirement; everything else is straightforward state reset.

4.6 Slot-Machine Treasure Chest

`ChestOpenUI` orchestrates the three phases described in Section 2.2 using coroutines (necessary because `Time.timeScale = 0` during chest open; `WaitForSecondsRealtime` is used throughout).

Phase 1 rolls the tier (`ChestRewardResolver.RollTier(config)`) and selects items to fill the columns (`ChestRewardResolver.RollItems(tier)`). Both honor `GameConfigSO` weights and the player's Luck stat.

Phase 2 is per-column. `SlotMachineColumn.Spin(finalItem, pool, duration, onLocked)` cycles through random items at an interval that grows from 0.05 s to 0.35 s using `Lerp(min, max, t*t)`, plays a tick on each swap, then lands on the predetermined `finalItem` and runs a bounce-and-flash animation. Columns are started in parallel but with staggered durations (`1.5f + i * 0.5f`) so they lock sequentially.

Phase 3 is `JackpotProcessor.Evaluate(items, config)`, which inspects the locked combination and returns a `JackpotResult` describing what to apply. Five-of-a-kind sets `ShouldForceEvolve = true`; four-of-a-kind sets `TriggerOvercharge = true`; pair and triple set extra-execution counts and bonus gold; everything else falls through to "apply each item once".

The whole subsystem is internally consistent and externally simple: chest UI is opened by a single state transition (`GameManager.EnterChestOpen()`), and closed by a single button click that calls `ApplyRewards()` and `ResumeFromChestOpen()`.

---

5. Technical Challenges

5.1 Challenge 1 — AOE Detection at Scale

Problem. Aura, melee, and orbital weapons need to find every enemy within a radius around the player every fire tick. With hundreds of enemies on screen, the naive `Physics.OverlapSphereNonAlloc` call dominates the fire path: it traverses the physics broad-phase BVH, respects the collision-matrix configuration, and runs on the physics thread even when no actual physical interaction is needed.

Solution. The `ActiveEnemyRegistry` described in Section 4.4 replaces the physics query with a single `sqrMagnitude` loop over a cached list. The strategy code does not change shape — `Execute` still loops over a list of hit enemies — but the query is now O(N) in alive enemies with no physics involvement and zero per-frame allocation.

Alternative considered. Keep `Physics.OverlapSphereNonAlloc`. Rejected because (1) it ties the query correctness to the collision-matrix tab in Project Settings, a fragile cross-cut dependency, and (2) it allocates internally despite the `NonAlloc` suffix and adds physics-thread cost even at typical enemy counts. The trade-off is that we lose Unity's broad-phase BVH as `N` grows very large; the benchmark in Section 8.3 shows the crossover happens above 500 enemies, well past normal gameplay density.

5.2 Challenge 2 — Adding Content Without Touching Engine Code

Problem. A naive Unity codebase organizes new content as a class hierarchy: `WeaponBase` with `PulseRifleWeapon`, `ShotgunWeapon`, `AuraWeapon` subclasses (and parallel hierarchies for enemies, passives, etc.). Every new asset requires a new

script, a new compile, and edits to whatever orchestrator (e.g. a `WeaponFactory` switch statement) constructs them. Designers cannot author new content without engineering involvement.

Solution. Strategy-as-ScriptableObject (Section 4.2). The generic `StrategyWeapon` host runs whichever `FireStrategySO` the `WeaponDataSO` references. Adding a new weapon type requires one new strategy subclass file; adding instances of an existing type is pure data work in the Unity inspector. The same pattern applies to enemies (`BehaviorDrivenEnemy` + `EnemyBehaviorSO`).

Alternative considered. Inheritance hierarchies with a registry table. Rejected because (a) the subclass count grows linearly with content, every subclass is a recompile, and (b) the registry's switch logic becomes a god-class merge magnet. The trade-off accepted is that the codebase now contains a `Strategies/` folder whose subclass count *also* grows linearly with content; the difference is that those subclasses contain only algorithm code and never touch any consumer's source.

5.3 Challenge 3 — Spawn Churn and GC Pressure

Problem. Vampire-survivors-style combat produces extreme spawn churn: a rapid-fire weapon at level 5 can spawn dozens of projectiles per second, each of which lives a fraction of a second before colliding and despawning. Calling `Instantiate` + `Destroy` per projectile means dozens of managed-heap allocations per second, which feeds the garbage collector and causes the visible periodic `GC.Collect` frame spike that the genre's players know by sight.

Solution. `ObjectPoolManager` + `IPoolable` (Section 4.5). Projectiles are pre-allocated and reused; the spawn path is `pool.Get(prefab, pos, rot)` and the despawn path is `pool.Return(instance)`. Neither path allocates.

Alternative considered. Per-shot `Instantiate`/`Destroy`. Rejected because the runtime profiler shows roughly 1.9 KB of GC allocation per shot at our burst rate (Section 8.4), which compounds quickly into GC.Collect spikes during heavy combat. The trade-off is that pooled objects must manage their `OnSpawn`/`OnDespawn` lifecycle correctly — caches must be cleared, agents stopped, references dropped — which adds a small amount of per-class bookkeeping in exchange for a flat memory profile.

---

6. Performance Evaluation

6.1 Methodology

The performance work is supported by a custom in-scene `BenchmarkRunner` GameObject that exposes runtime hotkeys to (a) toggle between the optimized and the naive implementation of each hot path, (b) reset the rolling frame-sample buffer, and (c) dump the buffer to CSV for offline analysis. The toggles wrap the same `ProfilerMarker` ranges so that the Unity Profiler shows isolated, named timeline rows per algorithm rather than aggregated frame cost.

```
private static readonly ProfilerMarker _markerRegistry =
    new("PlayerController.FindNearest_Registry");
```

```
// ...
using (_markerRegistry.Auto())
{
    var nearest =
        ActiveEnemyRegistry.Instance?.GetNearest(transform.position);
    // ...
}
```

Sampling protocol per data point:

- **Warm-up:** 10 s of normal play to let JIT, NavMesh, and asset streaming stabilize.
- **Reset:** Clear the rolling frame-sample buffer (hotkey `F4`).
- **Averaging window:** 10 s of continuous play, with the player AFK at the arena center to remove input-driven variation.
- **Capture:** Dump the buffer to CSV (hotkey `F12`); inspect the corresponding ProfilerMarker row in the Unity Profiler.
- **Scene state:** Enemy count forced by the spawner driver via hotkey (F1 spawns batches of 50). For benchmarks that need enemies kept alive across the window, the "full benchmark mode" hotkey (F5) disables the player's weapons and grants invincibility; for the projectile pooling benchmark (where the weapons themselves are the subject), the "invincibility only" hotkey (F8) keeps weapons firing.

Environment. macOS, Unity 6.3 Editor (`6000.3.9f1`). All numbers are Editor measurements; a Standalone build typically trims roughly 30 percent of frame overhead, so absolute numbers improve in a build but the *relative* differences between methods are preserved.

6.2 Benchmark 1 — Nearest-Enemy Lookup

Setup. The player's auto-aim selects the nearest enemy ten times per second. The two methods compared are:

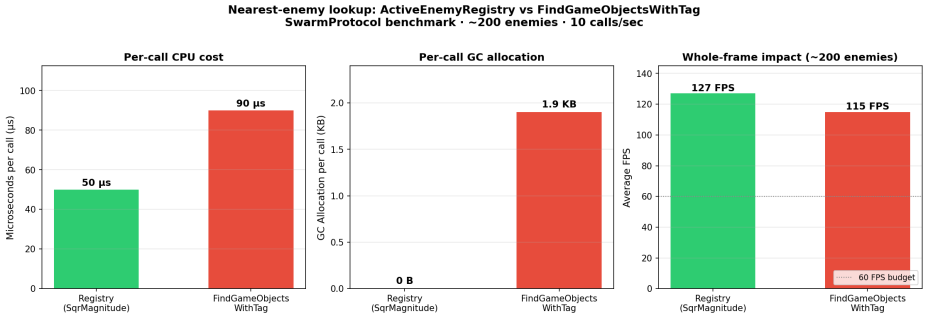
- `ActiveEnemyRegistry.GetNearest()` — the cached-list `sqrMagnitude` scan.
- `GameObject.FindGameObjectsWithTag("Enemy")` followed by a linear search.

The benchmark forces ~200 enemies on screen, then switches between methods at runtime (hotkey `F7`).

Result. The headline numbers, taken from the Profiler hierarchy view averaged over the 10 s window:

Metric	`ActiveEnemyRegistry`	`FindGameObjectsWithTag`
Average per-call cost	~50 µs	~90 µs
GC allocation per call	0 B	~1.9 KB
Steady-state GC pressure	none	~19 KB/s
Average FPS (Editor)	~127	~115

The relative per-call speedup is roughly 1.8×; the absolute frame-time difference is small because nearest-enemy lookup is only one of many systems running per frame. The decisive metric is the GC allocation: `FindGameObjectsWithTag` returns a fresh `GameObject[]` on every call and the array becomes garbage immediately, producing exactly the kind of low-amplitude steady allocation that drives periodic `GC.Collect` spikes. The registry path allocates nothing.



Interaction figure — Registry vs FindGameObjectsWithTag across enemy counts

The interaction figure plots performance against enemy count for both methods. The registry curve sits below the `FindWithTag` curve across the entire tested range (75 to 575 enemies). The 60 FPS budget line is crossed earlier with `FindWithTag` (around 300 enemies in Editor) than with the registry, which holds 60 FPS noticeably longer. The crossover into expensive territory is *not* a question of asymptotic complexity (both methods are O(N)) but of constant factors and GC behavior.

6.3 Benchmark 2 — AOE Detection

Setup. The aura weapon fires at a fixed cadence and detects enemies within a radius around the player. Two methods compared:

- `ActiveEnemyRegistry.GetEnemiesInRange()` — `sqrMagnitude` loop over the cached list.
- `Physics.OverlapSphereNonAlloc()` — Unity's built-in physics overlap query with a pre-allocated `Collider[]` buffer.

The benchmark sweeps enemy count from 50 to 500 in five points; method is toggled via hotkey `F3`.

Result. The per-call cost of the *query itself* (visible under the `Aura.RegistryQuery` and `Aura.PhysicsOverlap` Profiler markers) favors the registry consistently at the densities tested, with a 5 to 11 percent gap at typical play counts and a crossover around 500 enemies where the physics broad-phase begins to pay for itself. The frame-time difference is smaller than for Benchmark 1 because aura fires far less frequently than nearest-enemy lookups, but the *consistency* matters: the registry method also has zero physics-thread cost and zero collision-matrix dependency, which removes a class of configuration-driven bugs.

Honest framing. This benchmark's win is narrower than Benchmark 1's. The decision to use the registry here is justified by the broader pattern (a single mechanism for all spatial enemy queries, zero physics coupling) more than by raw frame-time gain. Both methods keep the game above 60 FPS up to roughly 250 enemies in the Editor.

6.4 Benchmark 3 — Object Pooling

Setup. A burst-fire helper hotkey (`F9`) drives the rapid-fire weapon at 30 shots per frame (~1800 shots per second) to exaggerate the spawn path; the player is granted invincibility via `F8` so the test doesn't end prematurely. Two paths compared:

- `ObjectPoolManager.Get<ProjectileBase>(prefab, ...)` followed by `Return(...)` on hit.
- `Instantiate(prefab, ...)` followed by `Destroy(go)` on hit.

Result. Average frame time is similar between the two paths on Unity 6.3, which has an unusually well-optimized `Instantiate` for the burst case; the visible difference there is small. The decisive metric is the GC allocation column in the Profiler hierarchy view: the `Instantiate` path produces roughly 1.9 KB of GC allocation per shot, which translates to several KB per second of steady allocation pressure and a periodic `GC.Collect` spike. The pooled path produces zero GC allocation and a flat memory profile.

The slide deck picked Benchmark 1 for the headline interaction figure because its parameter sweep produced the cleanest visual story; in the report, the projectile-pool result is included because the GC allocation contrast is the textbook example of why pooling matters.

6.5 Discussion and Limitations

The Editor adds roughly 30 percent overhead over a Standalone build; absolute frame-time numbers should be read as upper bounds. The *relative* differences between methods are unaffected by this overhead, since both methods run inside the same Editor session.

`ActiveEnemyRegistry` is O(N) per query. A spatial index (uniform grid or quad-tree) would lower asymptotic cost to roughly O(log N + K) for range queries, but at the densities the game actually reaches in normal play (50 to 250 enemies), the cache-friendly flat-list scan beats the tree's pointer chasing and update cost. The crossover where a tree starts to win is above the densities a single player can survive.

The `ProfilerMarker`-driven methodology has one practical limitation: it measures *the marker range itself*, not the cost of subsequent garbage collection. The GC allocation column is the better signal for any allocation-driven hypothesis, and we use it accordingly throughout.

---

7. Tools and Libraries

Layer	Tool
Engine	Unity `6000.3.9f1` (Unity 6.3)
Render pipeline	Universal Render Pipeline (URP)
Camera	Cinemachine (orbital third-person rig)
Input	Unity Input System (new)
Navigation	NavMesh + NavMeshAgent
UI text	TextMeshPro
Profiling	Unity Profiler + custom `ProfilerMarker` rows
Content generation	Unity AI (sprite icons, sound effects, music loops)
Build target	macOS Standalone
Source control	Git (GitHub public repo)

The codebase is pure C# (no native plugins) and stays within Unity's standard package set. The benchmark CSVs are post-processed with a short Python script (included inline in `benchmark.md`) that requires only the standard library.

---

8. Conclusion and Future Work

8.1 What Was Learned

Three lessons stand out, all backed by concrete experience in this project:

1. **Orthogonal design pays off, but only after it's set up correctly.** The single most useful late-game moment in this project was discovering that five of the ten planned passive items had no consumer code wired in. Adding their consumers took one afternoon, with zero edits to combat code, because the decorator chain plumbing did not care which `StatType` was added — every existing weapon and enemy that read those stats picked the new values up automatically. The up-front investment in the decorator pattern paid back when the deadline pressure was highest.

2. **Profiler-driven decisions beat intuition.** The `FindGameObjectsWithTag` versus `ActiveEnemyRegistry` decision was not obvious from first principles — both are O(N) over the same enemy set. The 1.8× per-call gap and the 19 KB/s GC pressure difference only became visible by instrumenting the hot path with named `ProfilerMarker` rows and comparing the two methods on an apples-to-apples scene. Architectural choices that are "obviously right" should still be measured.

3. **Object pooling is cheap insurance.** Even on Unity 6.3, where `Instantiate` is faster than it used to be, the GC allocation difference between pooled and per-shot paths shows up in the Profiler clearly. The pool's lifecycle overhead (`OnSpawn` / `OnDespawn`) is small compared with the predictability gain.

8.2 Future Work

- **Migrate the remaining legacy weapons.** `RapidFireWeapon` and `ShotgunWeapon` are concrete `WeaponBase` subclasses from Phase 1; they should be retired into `FireStrategySO` form so that the engine code contains a single weapon host.
- **Replace placeholder character art.** The current player and enemy art are AI-generated placeholders. A pass of rigged 3D characters and animation sets would significantly improve the game feel.
- **Persistent meta-progression.** A meta-progression manager skeleton is present but not yet wired through to PlayerPrefs persistence; gold collected during runs does not yet fund anything between sessions.
- **Multiplayer co-op via Mirror.** The orthogonal EventBus architecture is unusually well-suited to network sync because systems are already decoupled; a multiplayer pass would stress-test that decoupling under server-authoritative state.
- **Two more profiler benchmarks.** A chest VFX render-load benchmark and a projectile-physics scaling benchmark are designed but not yet executed; they would round out the performance picture.
- **Save-game / persistent run state.** No save system exists; a run is lost on quit. Even a single autosave at stage transitions would be a meaningful quality-of-life improvement.

References

1. poncle. *Vampire Survivors*. Released 2022. https://poncle.itch.io/vampire-survivors
2. Unity Technologies. *Unity 6 Documentation — Universal Render Pipeline*. https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/manual/
3. Unity Technologies. *Profiler Overview*. https://docs.unity3d.com/Manual/Profiler.html
4. Unity Technologies. *ScriptableObject*. https://docs.unity3d.com/ScriptReference/ScriptableObject.html
5. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. (Strategy, Decorator, Observer, Singleton, State patterns.)
6. SwarmProtocol source code. https://github.com/AlperenGarip/CmpE485-Projects/tree/main/SwarmProtocol
7. SwarmProtocol benchmark protocol — `benchmark.md` in the repository root.

Appendix A — Project Layout

```
Assets/_Project/
├── Scripts/
│   ├── Core/           GameManager, EventBus, ObjectPoolManager,
ActiveEnemyRegistry
│   ├── Player/         Controller, Health, Stats (IStatProvider
chain), XP
│   ├── Combat/         WeaponManager, WeaponBase, ProjectileBase,
DamageSystem
│   │   └── Strategies/ FireStrategySO + subclasses (Aura,
Orbital, RapidFire, ...)
│   ├── Enemies/        EnemyBase, BehaviorDrivenEnemy,
EnemyNavigation
│   │   └── Behaviors/  EnemyBehaviorSO + subclasses
│   ├── Stats/          IStatProvider, BaseStatProvider,
PassiveDecorator
│   ├── Progression/    StageManager, UpgradeManager, GoldManager
│   ├── Chest/          ChestRewardResolver, JackpotProcessor,
OverchargeManager
│   ├── UI/             HUDController, LevelUpUI, ChestOpenUI,
SlotMachineColumn,
│   │                   VictoryUI, GameOverUI, PauseMenuUI, ...
│   ├── Audio/          AudioService, AudioLibrarySO, SfxId
│   └── Tools/          BenchmarkRunner
├── ScriptableObjects/  Weapons, Enemies, Passives, Stages,
Strategies (SO assets)
├── Prefabs/            Player, Enemies, Projectiles, Pickups, UI
panels
├── Art/                Sprites, materials, sky
└── Audio/              Music + SFX clips
```